

main

October 4, 2025

```
[1]: import pandas as pd
import numpy as np
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score, cross_val_predict
from sklearn.metrics import r2_score

import matplotlib.pyplot as plt
# Load in the dataset
df = pd.read_excel("downsample_correct_columns.xlsx")

# Drop rows with missing values
df = df.dropna(subset=[
    "Incident State",
    "Date Of Incident",
    "Quantity Released",
    "Unit Of Measure",
    "Hazardous Class",
    "Time Of Incident",
    "Mode Of Transportation",
    "Total Evacuation Hours",
    "Public Evacuated",
    "Major Artery Hours Closed",
    "Undeclared Hazmat Shipment Ind"
])

# Convert date to datetime and pull month
df["Date Of Incident"] = pd.to_datetime(df["Date Of Incident"])
df["Incident Month"] = df["Date Of Incident"].dt.month

# clean mixed data column of hazardous class
df = df[df["Hazardous Class"].apply(lambda v: not isinstance(v, str))]
# 1. Classify 'Date Of Incident' as season
def get_season(month):
```

```

    if month in [12, 1, 2]:
        return 'Winter'
    elif month in [3, 4, 5]:
        return 'Spring'
    elif month in [6, 7, 8]:
        return 'Summer'
    else:
        return 'Fall'

df['Season'] = df['Date Of Incident'].dt.month.apply(get_season)

# 2. Classify 'Time Of Incident' as time of day
def get_time_of_day(time):
    # Convert to int in case it's float (e.g., 1330.0 -> 1330)
    t = int(time)
    hour = t // 100
    if 5 <= hour < 12:
        return 'Morning'
    elif 12 <= hour < 14:
        return 'Noon'
    elif 14 <= hour < 18:
        return 'Afternoon'
    elif 18 <= hour < 22:
        return 'Evening'
    else:
        return 'Night'

df['Time Of Day'] = df['Time Of Incident'].apply(get_time_of_day)
df=df.drop(['Date Of Incident', 'Time Of Incident'], axis=1)

# One-hot encode categorical variables
X_encoded = pd.get_dummies(df)

# Remove target from features
target = 'Total Amount Of Damages'

# Prepare data
X_features = X_encoded.drop(columns=[target])
y = X_encoded[target]

# Fit the pipeline on the whole data
lg=LinearRegression(fit_intercept=True)
lg.fit(X_features, y)

# Predict on the same data
y_pred = lg.predict(X_features)

```

```
# Report R2 score
r2 = r2_score(y, y_pred)
print(f"R2 score on training data: {r2:.3f}")
```

R2 score on training data: 0.403

```
[2]: plt.figure(figsize=(8, 6))
plt.scatter(y, y_pred, alpha=0.5)
plt.xlabel("Actual Total Amount Of Damages")
plt.ylabel("Predicted Total Amount Of Damages")
plt.title(f"Actual vs Predicted Damages (R2={r2:.2f})")
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--', lw=2)
plt.grid(True)
plt.tight_layout()
plt.show()
```

